SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Program Name:B. Tech	Assignment Type: Lab	Academic Year:2025-2026

Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s)Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju Mr. G. Kranthi

Course Code	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/I	**Regulation**	R23
Date and Day of Assignment	Week 7 - Thursday	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All Batches

AssignmentNumber:13.4 (Present assignment number)/**24**(Total number of assignments)

NAME : ERUKULLA MAHANTH ARYA

HALL TICKET : 2303A54028

BATCH -47A

Q.No.	Question	*ExpectedTime to complete*
1	**Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions** **Lab Objectives** • Identify code smells and inefficiencies in legacy Python scripts. • Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance. • Apply **modern Python best practices** while ensuring output correctness. **Learning Outcome** After completing this assignment, students will be able to: • Identify refactoring opportunities in legacy Python code • Apply AI-assisted suggestions to refactor code without changing behavior • Refactor Python code using Pythonic and scalable constructs • Critically evaluate AI-generated refactoring recommendations	Week 7

s	**Task 1: Refactoring Data Transformation Logic**
	
	Scenario
	
	You are maintaining a **data preprocessing script** where numerical transformations are written using verbose loops.
	
	Task Description
	
	• Review the legacy code that computes transformed values using an explicit loop.
	• Use an AI tool to suggest a more Pythonic refactoring approach.
	• Refactor the code using list comprehensions or helper functions while preserving the output.
	
	Legacy Code
	
	`values = [2, 4, 6, 8, 10]`
	`doubled = []`
	`for v in values:`
	`  doubled.append(v * 2)`
	`print(doubled)`	

Expected Output

```
[4, 8, 12, 16, 20]
```

PROMPT : ***I have the following legacy Python code that uses an explicit loop to double each value in a list: values = [2, 4, 6, 8, 10] doubled = [] for v in values: doubled.append(v * 2) print(doubled) Can you refactor this code using a more Pythonic approach such as a list comprehension or the map() function? Please preserve the original output [4, 8, 12, 16, 20] and explain the improvements made***

CODE :-

```
sing a list comprehension:
``python
alues = [2, 4, 6, 8, 10]
oubled = [v * 2 for v in values]
rint(doubled)
``Using the `map()` function:
``pythonvalues = [2, 4, 6, 8, 10]
oubled = list(map(lambda v: v * 2, values))
rint(doubled)
```

OUTPUT :

```
Expected Output

Legacy Code:
[4, 8, 12, 16, 20]
```

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a **log message generator** that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

PROMPT : Here is a legacy Python script that constructs a sentence using repeated string concatenation inside a loop: words = ["Refactoring", "with", "AI", "improves", "quality"] message = "" for w in words: message += w + " " print(message.strip()) String concatenation in a loop is inefficient in Python because strings are immutable, creating a new object at every iteration. Can you suggest and show a more efficient and readable alternative that produces the same output: "Refactoring with AI improves quality"?

CODE :

```
words = ["Refactoring", "with", "AI", "improves",
"quality"]



# Using str.join() — efficient and readable

message = " ".join(words)

print(messagE)
```

Output

```
Legacy Code:
Refactoring with AI improves quality
```

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a **configuration loader** where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}

if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

AI Prompt:

The following Python code manually checks if a key exists in a dictionary before accessing it: config = {"host": "localhost", "port": 8080} if "timeout" in config: print(config["timeout"]) else: print("Default timeout used") Can you refactor this code using a safer and more Pythonic dictionary access method? The code should handle the case where the "timeout" key is missing and still print "Default timeout used". Please explain why the refactored approach is preferred in production configuration loaders

CODE :-

Refactored Code:

```
config = {"host": "localhost", "port": 8080}

# Using dict.get() with a default value — safe and concise
print(config.get("timeout", "Default timeout used"))
```

Expected Output

```
Default timeout used
```

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a **utility module** that performs different operations based on user input.

Task Description

- Examine the multiple if–elif conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

AI Prompt:

*I have legacy Python code that uses multiple if-elif conditions to handle different arithmetic operations: action = "divide" x, y = 10, 2 if action == "add": result = x + y elif action == "subtract": result = x - y elif action == "multiply": result = x * y elif action == "divide": result = x / y else: result = None print(result) This approach becomes hard to maintain as more operations are added. Can you refactor this using a dictionary mapping technique with lambda functions or the operator module? The output must remain 5.0. Please explain how this makes the code more scalable*

Refactored Code:

```
import operator

# Dictionary mapping actions to operator functions
operations = {
    "add":      operator.add,
    "subtract": operator.sub,
    "multiply": operator.mul,
    "divide":   operator.truediv,
}

action = "divide"
```

Legacy Code

```
action = "divide"
x, y = 10, 2

if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
```

```
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None

print(result)
```

Expected Output

```
5.0
```

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing **legacy inventory-check code** that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False

for item in inventory:
    if item == "eraser":
        found = True
        break

print("Item Available" if found else "Item Not Available")
```

PROMPT :AI Prompt:
I have legacy Python code that uses multiple if-elif conditions to handle different arithmetic operations: action = "divide" x, y = 10, 2 if action == "add": result = x + y elif action == "subtract": result = x - y elif action == "multiply": result = x * y elif action == "divide": result = x / y else: result = None print(result) This approach becomes hard to maintain as more operations are added. Can you refactor this using a dictionary

mapping technique with lambda functions or the operator module? The output must remain 5.0. Please explain how this makes the code more scalable

CODE :-

Refactored Code:

```
inventory = ["pen", "notebook", "eraser", "marker"]

# Using 'in' membership operator — concise and readable
found = "eraser" in inventory
print("Item Available" if found else "Item Not Available")
```

Expected Output

```
Item Available
```

-

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots